

Platform Take Home Challenge

Designing a Secure, Developer-Friendly Release Pipeline

Contents

1	Executive Summary	2
2	Architecture Overview	2
2.0.1	Platform Infrastructure Repository	2
2.0.2	Application Repository	3
2.0.3	GitOps Repository	3
3	End-to-End Release Pipeline	3
3.1	Step 1 – Development	3
3.2	Step 2 – Artifact Creation	4
3.3	Step 3 – GitOps Promotion	4
3.4	Step 4 – Continuous Delivery	4
4	Environment Promotion Strategy	5
5	Hosting Platform	5
6	Security	6
6.0.1	Identity and Access Management	6
6.0.2	CI/CD Security Gates	7
6.0.3	Container Security	7
6.0.4	Kubernetes Workload Security	7
6.0.5	Infrastructure Security	7
6.0.6	Secrets Management	8
6.0.7	Kubernetes Runtime Security (Production Enhancement)	8
6.0.8	Production Deployment Security Gate	8
7	Observability	9
8	Reliability	9
8.1	Rollback Strategy	10
9	Developer Experience	10

10 Design Trade-offs	10
11 Conclusion	11

1 Executive Summary

This proposal describes a secure, scalable, and developer-friendly release pipeline designed to reduce deployment friction while maintaining strong operational controls. The solution follows GitOps principles and leverages Infrastructure as Code (IaC), automated testing, immutable artifacts, and progressive environment promotion to deliver reliable software changes from development to production.

The platform is built around the following technologies:

- Terraform for infrastructure provisioning
- Amazon Web Services (AWS)
- Amazon EKS as the hosting platform
- Docker for containerization
- GitHub Actions for Continuous Integration
- Amazon ECR for immutable artifact storage
- ArgoCD for GitOps-based Continuous Delivery
- Helm for Kubernetes application packaging

The design emphasizes automation, security, repeatability, and a positive developer experience while maintaining production-grade deployment controls.

2 Architecture Overview

The platform is separated into three independent repositories, each with a single responsibility.

2.0.1 Platform Infrastructure Repository

Responsible for provisioning cloud infrastructure using Terraform.

Infrastructure components include:

- Amazon VPC
- Amazon EKS
- Amazon ECR
- IAM Roles and Policies
- Networking

Separating infrastructure from application code enables independent lifecycle management, version control, and reproducible environments.

2.0.2 Application Repository

The application repository contains:

- Application source code
- Unit tests
- Integration tests
- Functional tests
- Regression tests
- End-to-end tests
- Performance tests
- Security tests
- Dockerfile
- CI pipeline

Developers interact only with this repository during feature development.

2.0.3 GitOps Repository

The GitOps repository contains the deployment configuration for each environment.

It includes:

- Helm chart
- Environment-specific values
- ArgoCD Applications
- Promotion workflows
- Validation workflows

The application repository never deploys directly to Kubernetes. Instead, deployments occur only after the GitOps repository is updated with a new immutable image tag.

3 End-to-End Release Pipeline

3.1 Step 1 – Development

A developer opens a Pull Request.

GitHub Actions automatically executes the Continuous Integration pipeline.

The pipeline performs:

- Source checkout

- Dependency installation
- Static analysis
- Unit testing
- Docker image build
- Security validation
- Artifact publishing

Only successful builds are allowed to continue.

3.2 Step 2 – Artifact Creation

A Docker image is built using the application’s Dockerfile.

The image is tagged using the Git commit SHA, ensuring every deployment references an immutable artifact.

The image is then pushed to Amazon ECR.

Using immutable image tags guarantees that every environment deploys exactly the same software version.

3.3 Step 3 – GitOps Promotion

After the image is published, a repository dispatch event triggers the GitOps repository.

The appropriate environment values file is updated with the new image tag.

Only deployment configuration changes are committed to the GitOps repository.

This provides:

- Complete deployment history
 - Easy rollback
 - Full auditability
 - Separation between application code and deployment configuration
-

3.4 Step 4 – Continuous Delivery

ArgoCD continuously monitors the GitOps repository.

Whenever a new image tag is committed, ArgoCD automatically synchronizes the Kubernetes cluster.

Deployments occur declaratively through Helm.

Because Kubernetes is reconciled continuously, cluster drift is automatically corrected.

4 Environment Promotion Strategy

The release pipeline promotes software through four environments.

Development

- Automatic deployment
- Smoke testing
- Fast feedback

↓

Quality Assurance

- Functional testing
- Integration testing

↓

Staging

- End-to-end testing
- Regression testing
- Performance validation

↓

Production

- Manual approval
- Controlled release

Each environment must successfully complete its validation workflow before promotion to the next stage.

This minimizes deployment risk while maintaining rapid delivery.

5 Hosting Platform

The solution assumes Amazon Elastic Kubernetes Service (EKS) as the production hosting platform.

Applications are deployed using Helm charts managed by ArgoCD.

The Helm chart includes:

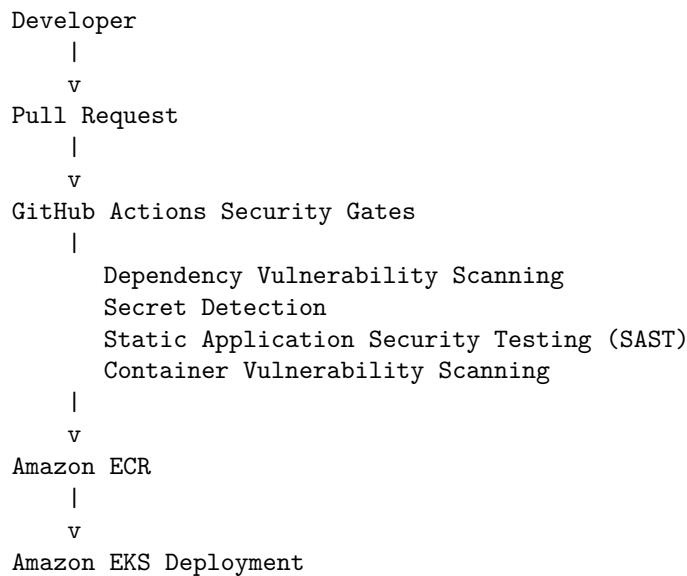
- Deployment
- Service
- Horizontal Pod Autoscaler
- Ingress

This allows applications to scale automatically while maintaining consistent deployments across every environment.

6 Security

Security is integrated throughout the release pipeline rather than added only at deployment time. The platform applies security controls across the entire software delivery lifecycle, including source code, CI validation, artifact management, infrastructure, and Kubernetes workloads.

The security model follows a defense-in-depth approach:



6.0.1 Identity and Access Management

GitHub Actions authenticates to AWS using OpenID Connect (OIDC) federation. This eliminates the need for long-lived AWS access keys and allows workflows to obtain temporary AWS credentials through IAM role assumption.

IAM permissions follow the principle of least privilege. The CI/CD IAM role is restricted to the permissions required to publish container images to Amazon

ECR and does not have unnecessary infrastructure or cluster administration permissions.

6.0.2 CI/CD Security Gates

Security validation is performed automatically during the CI pipeline before artifacts are promoted.

The pipeline includes:

- Dependency vulnerability scanning using pip-audit
- Secret detection using Gitleaks
- Static Application Security Testing (SAST) using Semgrep
- Container vulnerability scanning using Trivy

These checks provide early detection of security issues before software reaches deployment environments.

6.0.3 Container Security

Container images are secured through immutable artifact management and vulnerability scanning.

Amazon ECR is configured with:

- Immutable image tags to prevent artifact replacement
- Automated image scanning on push
- Versioned images using Git commit SHA identifiers

This ensures every deployment references a unique and traceable container artifact.

6.0.4 Kubernetes Workload Security

Kubernetes workloads apply container-level security hardening.

Security controls include:

- Running containers as non-root users
- Disabling privilege escalation
- Dropping unnecessary Linux capabilities
- Using read-only container filesystems
- Defining CPU and memory resource limits

These controls reduce the impact of potential container compromise and improve workload isolation.

6.0.5 Infrastructure Security

Infrastructure is provisioned and managed using Terraform.

The AWS environment includes:

- Dedicated VPC networking
- Public and private subnet separation
- Kubernetes worker nodes deployed in private subnets
- Multi-Availability Zone deployment
- IAM-based access management

Infrastructure changes are version controlled and reproducible, reducing configuration drift and improving operational consistency.

6.0.6 Secrets Management

Secrets are not stored in application repositories or GitOps configuration.

For production environments, AWS Secrets Manager integrated with Kubernetes External Secrets Operator would be used to securely deliver secrets to workloads.

This approach provides:

- Centralized secret management
- Improved access control
- Secret rotation capabilities
- Prevention of sensitive data exposure in Git repositories

6.0.7 Kubernetes Runtime Security (Production Enhancement)

Additional Kubernetes security controls would be introduced for production environments, including:

- Kubernetes RBAC for least-privilege cluster access
- Network Policies to restrict pod-to-pod communication
- Pod Security Standards to prevent insecure workloads
- IAM Roles for Service Accounts (IRSA) for secure AWS service access from Kubernetes workloads

6.0.8 Production Deployment Security Gate

Production deployments require successful validation in lower environments before promotion.

The release flow follows:

DEV → QA → STAGING → Production Approval → PRODUCTION

The production environment would use deployment protection rules including:

- Required reviewer approval
- Deployment audit history
- Restricted production access
- Separation of duties

This prevents unvalidated changes from reaching production while maintaining a controlled and auditable release process.

7 Observability

The platform includes an observability stack to monitor application health, infrastructure performance, and deployment reliability.

The solution uses:

- Prometheus for metrics collection
- Grafana for dashboards and alerting
- Fluent Bit to collect container logs
- CloudWatch Logs or Loki for centralized log storage
- ArgoCD and Kubernetes health checks to monitor deployments

Key metrics include:

- Request rate and latency
- Error rate
- Pod health and restarts
- CPU and memory utilization
- Kubernetes deployment status

Alerts are generated for conditions such as:

- Application failures
- High error rates
- Increased latency
- Pod crash loops
- Resource exhaustion
- Node failures

For distributed applications, OpenTelemetry can be added to provide end-to-end request tracing across services.

This observability stack provides engineers with rapid feedback, simplifies troubleshooting, and helps detect deployment issues before they impact users.

8 Reliability

Platform reliability is achieved through several architectural decisions:

- Immutable Docker images eliminate configuration drift.
- GitOps continuously reconciles the desired cluster state.
- Multi-stage environment promotion reduces production risk.

- Terraform provides reproducible infrastructure.
- Multi-AZ EKS improves availability.

8.1 Rollback Strategy

Rollback is performed by reverting the image tag in the GitOps repository. ArgoCD automatically detects the change and restores the previous known-good version without requiring manual Kubernetes changes.

9 Developer Experience

The pipeline is designed to minimize manual work.

Developers only need to:

1. Commit code.
2. Open a Pull Request.
3. Merge after approval.

Everything else is automated.

This includes:

- Testing
- Image creation
- Artifact publishing
- Environment promotion
- Kubernetes deployment

Automation allows engineers to focus on application development instead of deployment mechanics.

10 Design Trade-offs

Several architectural decisions were made intentionally.

GitOps was selected over imperative deployment because it provides a complete deployment history, easier rollbacks, and automatic reconciliation.

Amazon EKS was selected because it provides a mature Kubernetes platform with strong scalability and ecosystem support.

Terraform was selected to ensure infrastructure is reproducible, version-controlled, and consistent across environments.

Helm simplifies Kubernetes resource management while enabling reusable deployment templates across multiple environments.

Separating infrastructure, application code, and deployment configuration into independent repositories increases maintainability and allows teams to work independently.

11 Conclusion

This design provides a secure, GitOps-based release pipeline that emphasizes automation, repeatability, and developer productivity. By combining Terraform, GitHub Actions, Amazon ECR, Helm, ArgoCD, and Amazon EKS, the platform delivers immutable deployments, automated validation, controlled environment promotion, and reliable continuous delivery with minimal operational overhead.